Birla Institute of Technology Mesra

Department of Quantitative Economics and Data Science

ED408: Statistical Machine Learning II Lab

Lab File — Module II

Optimisation, Regularisation and Kernel Methods

Datasets used:	Synthetic benchmarks; scikit-learn *diabetes*
Topics:	Metaheuristics, regularised regression, Kernel PCA
Experiments:	8–13
Submitted by:	Priyanshu Ranjan
Roll:	IED/10011/23

The complete executable programs are maintained digitally on GitHub;
this file records the important code sections, outputs and analysis.

Common Setup (used by all experiments)

Environment. Python 3 with `numpy`, `pandas`, `matplotlib` and `scikit-learn`. Every experiment is a standalone, fully executed notebook (`Lab08`–`Lab13` in the `module 2` folder).

Reproducibility. All random choices (cities, graphs, swarm initialisation, train/test data, ant moves) are driven by fixed `numpy` seeds, so every number in this file is reproduced exactly by re-running the notebook.

Two themes across the module.

- **Exploration vs exploitation (Exp. 8, 9, 10, 13).** Simulated Annealing, Genetic Algorithms, Particle Swarm Optimisation and Ant Colony Optimisation are all population- or trajectory-based metaheuristics. Each has one or more parameters that trade wide exploration against fast convergence: temperature schedule, mutation rate, inertia weight, pheromone evaporation and bounds. Every experiment varies that parameter and measures the effect.
- **Controlling model complexity (Exp. 11, 12).** Regularisation limits what a linear model can fit; Kernel PCA instead *widens* what a linear method can see by working in an implicit feature space. In both cases a single hyperparameter (α, γ) decides whether the method under- or over-fits.

Verification. Wherever an exact answer exists, the metaheuristic result is checked against it: a dense grid search (Exp. 8), the analytic optimum $(1, \ldots, 1)$ (Exp. 10), `sklearn.decomposition.KernelPCA` (Exp. 12) and Dijkstra's algorithm (Exp. 13).

Experiment No.: 08

Title: Simulated Annealing for Function Optimisation

Aim

Implement Simulated Annealing (SA) to minimise $f(x) = x^2 + 10\sin(x)$ on $[-10, 10]$, and compare it with greedy hill climbing.

Problem Statement

f has a *local* minimum near $x \approx 3.84$ ($f \approx 8.32$) and the *global* minimum near $x \approx -1.31$ ($f \approx -7.95$), separated by a hill at $x \approx 1.98$. A search that only accepts improvements gets trapped in whichever basin it starts in. Show that SA's probabilistic acceptance of uphill moves lets it escape, and study how the cooling rate affects reliability and cost.

Algorithm

1. Start at x_0 with temperature $T_0 = 10$; record the best-so-far solution.
2. At each temperature, repeat 20 times: propose a neighbour $x' = \text{clip}(x + \varepsilon, -10, 10)$ with $\varepsilon \sim \mathcal{N}(0, 0.5^2)$; compute $\Delta f = f(x') - f(x)$.
3. Accept with the Metropolis rule $P = 1$ if $\Delta f \leq 0$, otherwise $P = e^{-\Delta f / T}$; update the best-so-far solution.
4. Cool geometrically, $T \leftarrow \alpha T$ with $\alpha = 0.95$, until $T < T_{min} = 10^{-3}$.
5. Baseline: hill climbing with the same neighbour rule and evaluation budget, accepting only $\Delta f < 0$.
6. Verify against a 200,001-point grid search; repeat from 30 random starts for $\alpha \in \{0.80, 0.95, 0.99\}$.

Program (important sections)

```python
def simulated_annealing(f, x0, T0=10.0, alpha=0.95, T_min=1e-3,
                        n_per_T=20, step=0.5, rng=None):
    x, fx = x0, f(x0)
    best_x, best_f = x, fx
    T = T0
    while T > T_min:
        for _ in range(n_per_T):
            xn = np.clip(x + rng.normal(0, step), LO, HI) # random neighbour
            fn = f(xn)
            d = fn - fx
            if d <= 0 or rng.random() < np.exp(-d / T): # Metropolis rule
                x, fx = xn, fn
                if fx < best_f:
                    best_x, best_f = x, fx
        T *= alpha # geometric cooling
    return best_x, best_f
```

Key parameter settings: $T_0 = 10$, $\alpha = 0.95$, $T_{min} = 10^{-3}$, 20 moves per temperature (3600 evaluations), step $\sigma = 0.5$, start $x_0 = 8$.

Output and Analysis

Program execution:

```
Grid search: x* = -1.3064, f(x*) = -7.9458
Start x0 = 8.0, f(x0) = 73.8936, iterations = 3600
Simulated Annealing : x = -1.3063, f = -7.9458 (uphill moves accepted: 556)
Hill climbing : x = 3.8375, f = 8.3156
Execution Status: Successfully Executed
```

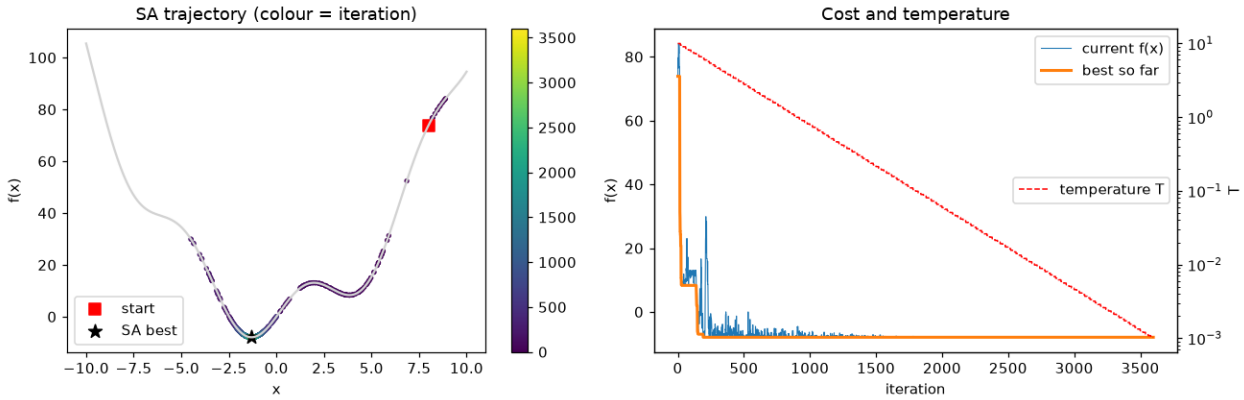


Figure 1: Left: SA trajectory on $f(x)$, coloured by iteration. Right: current and best-so-far cost, with the temperature (log scale) on the right axis.

Parameter modification (30 random starts; success = final $|x - x^*| < 0.1$):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Cooling rate α	0.95	0.80	100%, mean $f = -7.946$	80%, mean $f = -4.694$	Fast cooling freezes the walker before it has explored both basins.
2	Cooling rate α	0.95	0.99	100%, 3600 evals	100%, 18340 evals	Slower cooling costs 5× more evaluations for no gain here.
3	Acceptance rule	Metropolis	greedy only	100% success	43.3%, mean $f = 1.269$	Hill climbing succeeds only from the 13/30 starts already in the global basin.

Conclusion. SA finds the global minimum ($x = -1.3063$, $f = -7.9458$, matching the grid search) from a start on the wrong side of the hill, accepting 556 uphill moves on the way, nearly all at high temperature. The cost curve shows the escape: the best value sits on the local plateau ($f \approx 8.3$) for about 150 iterations, then drops once the walker crosses the hill. Below $T \approx 0.1$ the search is effectively greedy and only fine-tunes. The cooling rate is the key parameter: too fast and SA degenerates into hill climbing; slower than necessary only wastes evaluations.

Experiment No.: 09

Title: Genetic Algorithm for the Travelling Salesman Problem

Aim

Implement a Genetic Algorithm (GA) with permutation encoding to find a short closed tour through 25 cities.

Problem Statement

A tour is a permutation π of the cities with length $L(\pi) = \sum_{i=1}^{N} d(\pi_i, \pi_{i+1})$, $\pi_{N+1} = \pi_1$. There are $24!/2 \approx 3 \times 10^{23}$ distinct tours, so exhaustive search is impossible. Binary encoding and ordinary one-point crossover would create tours that repeat cities, so the GA needs permutation-safe operators. Compare the GA with random tours and the nearest-neighbour heuristic, and measure the contribution of each operator.

Algorithm

1. Place $N = 25$ cities uniformly in $[0, 100]^2$; precompute the distance matrix.
2. Initialise 150 random permutations. Fitness is $1/L(\pi)$ (equivalently, minimise L).
3. **Selection:** tournament of size 3 (the shortest of 3 random tours wins).
4. **Crossover** (prob. 0.9): Order Crossover (OX). Copy a random slice from parent 1, then fill the remaining positions with the missing cities in the order they appear in parent 2, starting after the slice.
5. **Mutation** (prob. 0.3): inversion, i.e. reverse a random sub-tour (a 2-opt move).
6. **Elitism:** copy the best 2 tours unchanged into the next generation; run 500 generations.

Program (important sections)

```
def order_crossover(p1, p2):
    n = len(p1)
    a, b = sorted(rng.choice(n, 2, replace=False))
    child = -np.ones(n, dtype=int)
    child[a:b+1] = p1[a:b+1]
    taken = set(p1[a:b+1])
    rest = [c for c in np.roll(p2, -(b+1)) if c not in taken] # p2 order
    child[[(b + 1 + i) % n for i in range(len(rest))]] = rest
    return child

def inversion_mutation(t, pm):
    t = t.copy()
    if rng.random() < pm:
        a, b = sorted(rng.choice(len(t), 2, replace=False))
        t[a:b+1] = t[a:b+1][::-1] # 2-opt move
    return t

for g in range(generations):
    lengths = np.array([tour_length(t) for t in pop])
    new = [pop[i] for i in np.argsort(lengths)[:elite]] # elitism
    while len(new) < pop_size:
        p1, p2 = tournament(pop, lengths), tournament(pop, lengths)
        child = order_crossover(p1, p2) if rng.random() < pc else p1.copy()
        new.append(inversion_mutation(child, pm))
    pop = np.array(new)
```

Key parameter settings: population 150, 500 generations, $p_c = 0.9$, $p_m = 0.3$, tournament size 3, elite 2.

Output and Analysis

Program execution:

```
p1 : [1 4 3 8 2 7 9 0 6 5]
p2 : [8 0 3 4 7 5 9 6 1 2]
child: [8 3 4 1 2 7 9 0 6 5] valid: True

Random tour : 1299.44
Nearest-neighbour (best start) : 510.88
Genetic Algorithm : 466.91
Execution Status: Successfully Executed
```

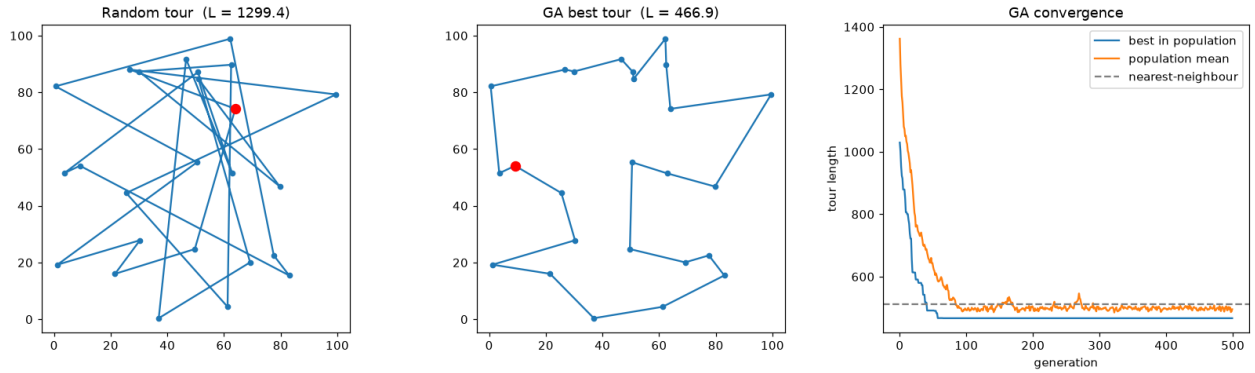


Figure 2: Left: a random tour. Centre: the GA's best tour (red dot = start city). Right: best and mean tour length per generation, with the nearest-neighbour tour for reference.

Parameter modification (mean final tour length over 5 seeds; baseline mean = 471.0):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Elitism	2 elites	none	mean $L = 471.0$	mean $L = 520.7$	Without elitism good tours are lost to unlucky crossovers.
2	Mutation p_m	0.3	0	mean $L = 471.0$	mean $L = 518.3$	No mutation: the population loses diversity and converges prematurely.
3	Crossover p_c	0.9	0	mean $L = 471.0$	mean $L = 486.3$	Mutation alone still works (it is a 2-opt local search), but recombination helps.
4	Population	150	50	mean $L = 471.0$	mean $L = 476.6$	Smaller populations are slightly less reliable; one run found $L = 464.7$.

Conclusion. The GA cuts the tour from 1299.4 (random) to 466.9, about 9% shorter than the best nearest-neighbour tour (510.9, tried from every start city), and the final tour has no crossing edges. Most of the improvement happens in the first ∼60 generations; after that the population mean sits just above the best tour, i.e. the population has converged. A single run is not guaranteed optimal: one 50-tour run found $L = 464.7$. Each operator matters: removing elitism or mutation costs about 10% in tour length.

Experiment No.: 10

Title: Particle Swarm Optimisation on the Rosenbrock Function

Aim

Apply Particle Swarm Optimisation (PSO) to minimise the Rosenbrock function, visualise the swarm, and study how the result scales with dimension and swarm parameters.

Problem Statement

$$f(\mathbf{x}) = \sum_{i=1}^{d-1} \Big[100\,(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\Big], \qquad \mathbf{x}^* = (1, \ldots, 1),\ f(\mathbf{x}^*) = 0.$$

The minimum lies at the end of a long, curved, almost flat valley. Finding the valley is easy, but moving along it to $(1, \ldots, 1)$ is slow, which makes Rosenbrock a standard optimisation benchmark.

Algorithm

1. Initialise n particles uniformly in the search box, with random velocities in $[-v_{max}, v_{max}]$, $v_{max} = 0.2\,(hi - lo)$.
2. Each particle remembers its personal best $\mathbf{p}_i$; the swarm shares the global best $\mathbf{g}$.
3. Update $\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1 r_1(\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2(\mathbf{g} - \mathbf{x}_i)$, with $r_1, r_2 \sim U(0,1)$ drawn per particle and per dimension; clamp to $\pm v_{max}$.
4. Move $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$, clip to the box, then update $\mathbf{p}_i$ and $\mathbf{g}$.
5. Decrease the inertia w linearly from 0.9 to 0.4: explore early, exploit late.
6. 2-D run (30 particles, 300 iterations) with snapshots; then $d \in \{2, 5, 10\}$ with 20 or 50 particles, 2000 iterations, 10 seeds each.

Program (important sections)

```
def rosenbrock(X): # X: (n_particles, d)
    return np.sum(100*(X[:, 1:] - X[:, :-1]**2)**2 + (1 - X[:, :-1])**2, axis=1)

for t in range(iters):
    w = w_max - (w_max - w_min) * t / iters # 0.9 -> 0.4
    r1, r2 = rng.random((2, n_particles, d))
    V = np.clip(w*V + c1*r1*(P - X) + c2*r2*(g - X), -v_max, v_max)
    X = np.clip(X + V, lo, hi)
    fx = f(X)
    better = fx < P_f # personal bests
    P[better], P_f[better] = X[better], fx[better]
    g = P[np.argmin(P_f)].copy() # global best
```

Key parameter settings: $c_1 = c_2 = 2.0$, $w : 0.9 \to 0.4$, $v_{max} = 20\%$ of the box width; 2-D box $[-2, 2] \times [-1, 3]$, higher-dimensional box $[-5, 10]^d$.

Output and Analysis

Program execution:

```
PSO best (d=2): x = [1.00001 1.00001], f = 3.251e-11 (true optimum (1, 1), f = 0)

 d particles median best f runs with f < 1e-4
 2 20 1.52e-30 10/10
 2 50 0.00e+00 10/10
 5 20 1.54e-01 0/10
 5 50 5.65e-02 0/10
10 20 3.49e+00 0/10
10 50 3.09e+00 0/10
```

```
Execution Status: Successfully Executed
```

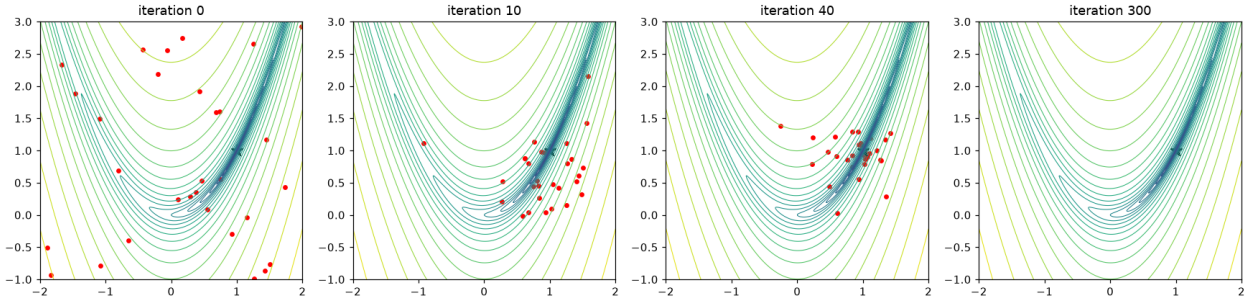


Figure 3: The swarm (red) on the log-scaled Rosenbrock contours at iterations 0, 10, 40 and 300; the star marks the optimum $(1, 1)$.

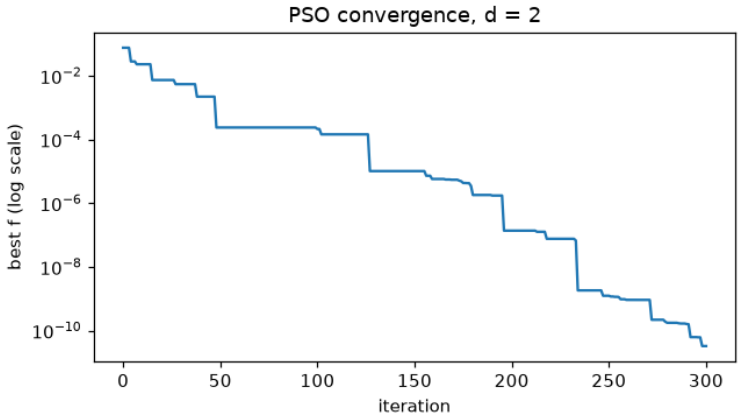


Figure 4: Best value found versus iteration (log scale), $d = 2$.

Parameter modification (rows 1–2: $d = 2$, 30 particles, 300 iterations, median of 10 seeds):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Inertia w	$0.9 \to 0.4$	constant 0.9	$f = 7.8 \times 10^{-10}$	$f = 1.2 \times 10^{-4}$	High inertia never damps the particles, so the swarm cannot settle.
2	Inertia w	$0.9 \to 0.4$	constant 0.4	$f = 7.8 \times 10^{-10}$	$f = 3.6 \times 10^{-14}$	In 2-D pure exploitation is enough; early exploration matters on harder landscapes.
3	Dimension d	2	10	median $f \approx 0$	median $f = 3.09$	gbest PSO collapses in the narrow valley and stalls before $(1, \ldots, 1)$.
4	Swarm size ($d = 10$)	20	50	median $f = 3.49$	median $f = 3.09$	More particles help only slightly; the topology, not the count, is the bottleneck.

Conclusion. In 2-D PSO locates the Rosenbrock minimum to $f \approx 3 \times 10^{-11}$. The snapshots show the two phases: by iteration 10 the swarm has gathered along the valley, and by iteration 40 it is moving along it towards $(1, 1)$. The inertia weight controls whether the swarm can settle at all. In 5-D and 10-D the global-best swarm finds the valley but not the optimum within 2000 iterations. This is the known weakness of gbest PSO on Rosenbrock: all particles collapse onto one point and lose the diversity needed to keep moving. Ring (local-best) topologies, constriction coefficients or restarts are the standard remedies.

Experiment No.: 11

Title: L1, L2 and Elastic Net Regularisation on a Regression Dataset

Aim

Demonstrate L1 (Lasso), L2 (Ridge) and Elastic Net regularisation on a regression dataset: their effect on test error, on sparsity and on the coefficient paths.

Problem Statement

With few samples and many correlated features, ordinary least squares (OLS) overfits. The three penalties (scikit-learn parameterisation) are

$$\text{Ridge: } \|y - X\beta\|^2 + \alpha\|\beta\|_2^2, \qquad \text{Lasso: } \tfrac{1}{2n}\|y - X\beta\|^2 + \alpha\|\beta\|_1,$$

$$\text{Elastic Net: } \tfrac{1}{2n}\|y - X\beta\|^2 + \alpha\rho\|\beta\|_1 + \tfrac{\alpha(1-\rho)}{2}\|\beta\|_2^2.$$

A synthetic dataset makes the true coefficients known: $p = 40$ correlated features ($\text{corr}(x_i, x_j) = 0.7^{|i-j|}$), only 7 truly non-zero, noise $\sigma = 2$, **60 training** and 2000 test samples. The comparison is then repeated on the real scikit-learn *diabetes* dataset (442 patients, 10 features).

Algorithm

1. Generate the data; standardise features with training statistics (penalties assume a common scale).
2. Fit OLS, and tune Ridge, Lasso and Elastic Net ($\rho \in \{0.1, \ldots, 0.9\}$) by 5-fold cross-validation.
3. Compare test MSE, R^2, the number of non-zero coefficients and how many true or noise features each model keeps.
4. Plot the estimated coefficients against the truth, and the full regularisation paths over $\alpha \in [10^{-3}, 10^{1.5}]$.
5. Repeat the CV comparison on the diabetes dataset (70/30 split).

Program (important sections)

```
models = {
    "OLS (no penalty)": LinearRegression(),
    "Ridge (L2)": RidgeCV(alphas=np.logspace(-3, 3, 100), cv=5),
    "Lasso (L1)": LassoCV(cv=5, max_iter=50000, random_state=0),
    "Elastic Net": ElasticNetCV(l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9], cv=5,
                                max_iter=50000, random_state=0),
}
for name, m in models.items():
    m.fit(X_tr, y_tr)
    nz = np.abs(m.coef_) > 1e-8 # selected features
    print(name, mean_squared_error(y_te, m.predict(X_te)), nz.sum(),
          (nz & true_support).sum())

# regularisation paths (alphas in descending order)
_, lasso_coefs, _ = lasso_path(X_tr, y_tr - y_tr.mean(), alphas=path_alphas)
_, enet_coefs, _ = enet_path(X_tr, y_tr - y_tr.mean(), l1_ratio=0.5,
                             alphas=path_alphas)
```

Output and Analysis

Program execution (synthetic data):

Model	CV α	Test MSE	Test R^2	Non-zero	True kept (of 7)	Noise kept (of 33)
OLS (no penalty)	—	22.021	0.470	40	7	33
Ridge (L2)	3.275	9.867	0.763	40	7	33
Lasso (L1)	0.102	**6.826**	**0.836**	25	7	18
Elastic Net ($\rho = 0.9$)	0.105	7.078	0.830	25	7	18

```
Diabetes (test set): OLS MSE = 3097.1 R2 = 0.393 non-zero = 10
                     Ridge MSE = 3075.2 R2 = 0.397 non-zero = 10
                     Lasso MSE = 3116.2 R2 = 0.389 non-zero = 7
                     ENet MSE = 3086.4 R2 = 0.395 non-zero = 10
Execution Status: Successfully Executed
```

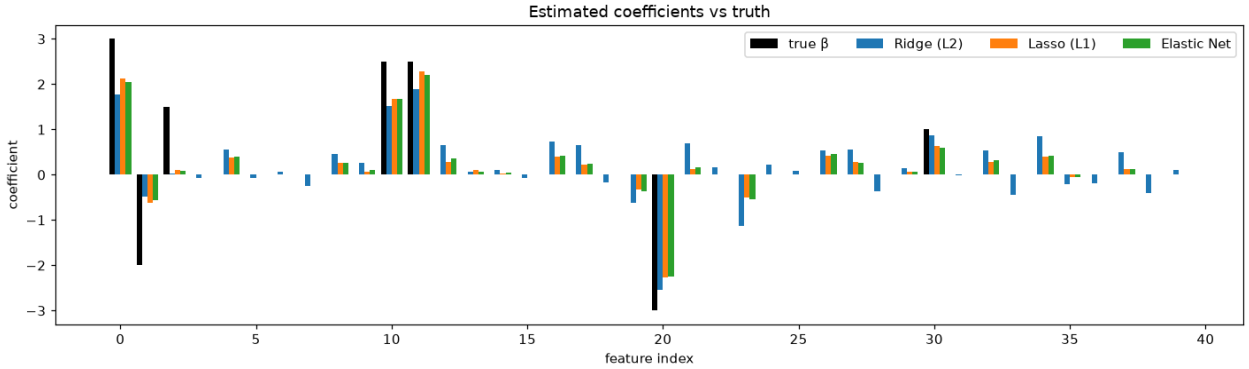


Figure 5: Estimated coefficients of the CV-tuned models against the true β (black).

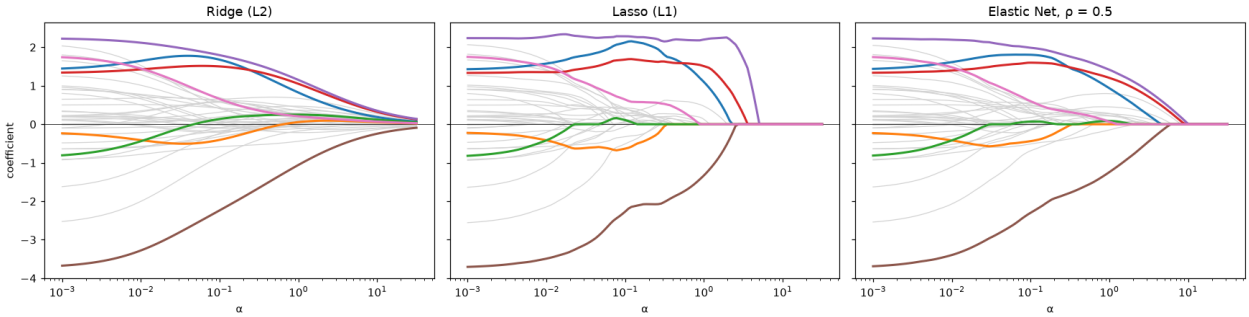


Figure 6: Regularisation paths: the 7 true features in colour, the 33 noise features in grey. Lasso coefficients hit exactly zero one by one; Ridge coefficients only shrink.

Parameter modification (synthetic data):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Penalty	L1 (Lasso, CV)	none (OLS)	MSE = 6.83	MSE = 22.02	With $n = 60$, OLS fits the r test error triple
2	Penalty	L1	L2 (Ridge, CV)	MSE = 6.83, 25 non-zero	MSE = 9.87, 40 non-zero	L2 shrinks eve cient but never to zero.
3	Lasso α	0.102 (CV)	1.0	MSE = 6.82, 7/7 true	MSE = 11.04, 4/7 true	Too strong a starts deleting tures.
4	Elastic Net ρ	0.9 (CV)	0.5	MSE = 7.08, 25 non-zero	MSE = 8.13, 28 non-zero	More L2 kee (correlated) and selects less

Conclusion. Every penalty beats OLS when n is small: Ridge raises test R^2 from 0.47 to 0.76, and Lasso and Elastic Net reach 0.83. Only the L1 penalty produces exact zeros: Lasso keeps all 7 true features and drops 15 of the 33 noise features. Cross-validation optimises prediction, not support recovery, so some small noise coefficients survive. True feature 2 is nearly zeroed by both L1 models because its correlated neighbours absorb its effect, the classic difficulty of L1 with correlated designs. On diabetes ($n \gg p$) all four models score the same ($R^2 \approx 0.39$–0.40), but regularisation still helps interpretation: Lasso removes `s2`, `s4` and `s6`, and every penalty shrinks the unstable collinear `s1`/`s2` pair (OLS: -23.9 and $+11.5$).

Experiment No.: 12

Title: Kernel PCA on Nonlinear Datasets

Aim

Implement Kernel PCA with an RBF kernel, verify it against scikit-learn, and visualise how it separates nonlinear data that linear PCA cannot.

Problem Statement

Linear PCA only rotates the data, so it cannot separate classes that are nested (concentric circles) or interlocked (two moons). Kernel PCA implicitly maps points to a feature space $\phi(\mathbf{x})$ and runs PCA there, using only the kernel matrix $K_{ij} = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$. Measure how well a *linear* classifier separates the classes after each projection, and how the result depends on γ.

Algorithm

1. Generate `make_circles` (600 points, factor 0.3, noise 0.05) and `make_moons` (600 points, noise 0.05).
2. Build the RBF kernel matrix K.
3. Centre it in feature space: $\tilde{K} = K - \mathbf{1}_n K - K\mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n$, with $(\mathbf{1}_n)_{ij} = 1/n$.
4. Eigendecompose $\tilde{K}$; the projection on component k is $\sqrt{\lambda_k}\,\mathbf{a}_k$.
5. Check against `sklearn.decomposition.KernelPCA` (correlation of components, up to sign).
6. Score linear PCA and Kernel PCA projections with 5-fold CV logistic regression; sweep γ from 0.1 to 1000.

Program (important sections)

```python
def kernel_pca(X, gamma, n_components=2):
    sq = np.sum((X[:, None] - X[None]) ** 2, axis=-1)
    K = np.exp(-gamma * sq) # RBF kernel matrix
    n = len(K)
    one = np.full((n, n), 1 / n)
    Kc = K - one @ K - K @ one + one @ K @ one # centre in feature space
    vals, vecs = np.linalg.eigh(Kc)
    top = np.argsort(vals)[::-1][:n_components]
    return vecs[:, top] * np.sqrt(vals[top]) # projected training points

ref = KernelPCA(n_components=2, kernel="rbf", gamma=gamma).fit_transform(X)
acc = cross_val_score(LogisticRegression(), Z, y, cv=5).mean()
```

Key parameter settings: $\gamma = 5$ (circles), $\gamma = 15$ (moons), 2 components.

Output and Analysis

Program execution:

```
Circles |corr(scratch, sklearn)| PC1 = 1.000000 PC2 = 1.000000
Moons |corr(scratch, sklearn)| PC1 = 1.000000 PC2 = 1.000000
Execution Status: Successfully Executed
```

Features given to a linear classifier	Circles	Moons
original x_1, x_2	0.480	0.882
linear PCA, PC1	0.487	0.728
linear PCA, PC1+PC2	0.480	0.882
Kernel PCA, PC1	**1.000**	**0.997**
Kernel PCA, PC1+PC2	**1.000**	**0.997**

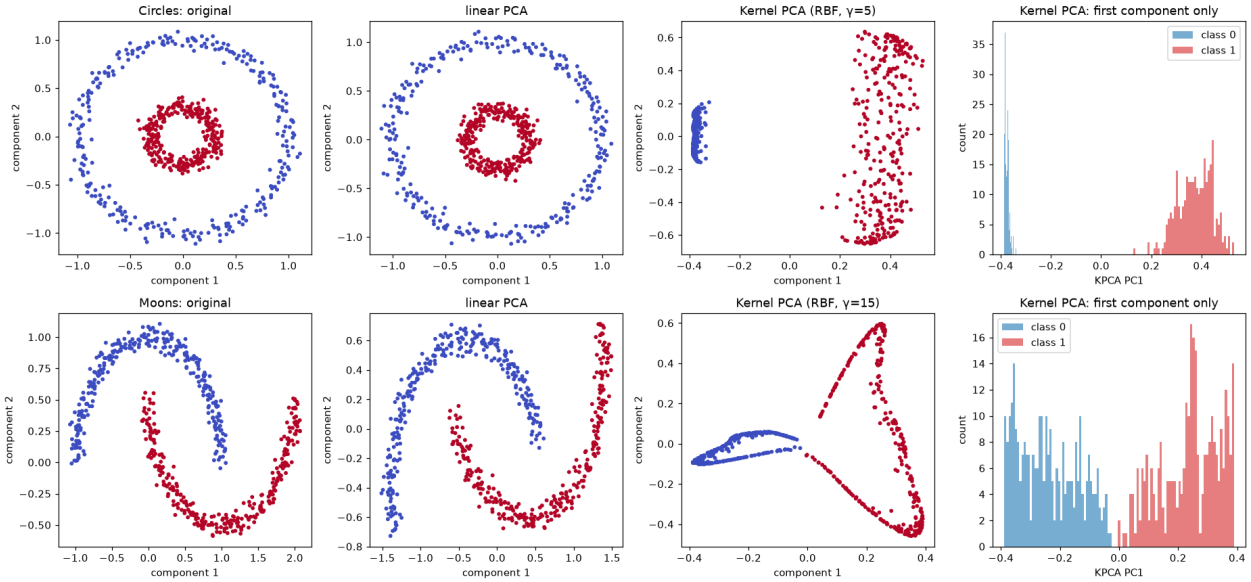


Figure 7: Circles (top) and moons (bottom): original data, linear PCA, Kernel PCA, and a histogram of the first kernel component by class.

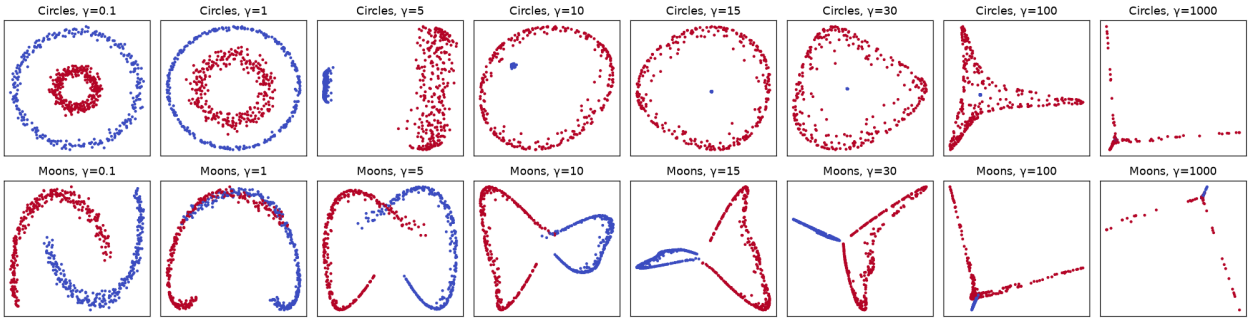


Figure 8: Kernel PCA projections for γ from 0.1 to 1000.

Parameter modification (linear-classifier accuracy on 2 kernel components):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Kernel (circles)	RBF, $\gamma = 5$	linear (PCA)	acc = 1.000	acc = 0.480	Linear PCA is only a rotation, so the rings stay nested (chance level).
2	γ (circles)	5	0.1	acc = 1.000	acc = 0.482	A very wide kernel is almost linear and behaves like plain PCA.
3	γ (circles)	5	1000	acc = 1.000	acc = 0.577	A very narrow kernel makes $K \approx I$: each point is similar only to itself.
4	γ (moons)	15	1	acc = 0.997	acc = 0.785	The best γ depends on the data's length scale (best range 15–30 here).

Conclusion. The from-scratch implementation matches scikit-learn exactly. Kernel PCA turns both nonlinear problems into linearly separable ones: on the circles the first kernel component alone separates the rings perfectly (the outer ring collapses to a tight cluster), and on the moons it gives 99.7% against 72.8% for linear PC1. Kernel PCA never saw the labels, so the separation appears only because the classes lie on different manifolds. γ is the critical hyperparameter and should be tuned by cross-validation on the downstream task.

Experiment No.: 13

Title: Ant Colony Optimisation for Shortest-Path Finding

Aim

Simulate Ant Colony Optimisation (ACO) to find the shortest path between two nodes of a weighted graph, and verify it against Dijkstra's algorithm.

Problem Statement

The network has 40 random junctions in the unit square, each linked to its 4 nearest neighbours (99 edges). The weight of an edge is its travel time $w_{ij} = d_{ij}\, c_{ij}$, where $c_{ij} \sim U(1, 3)$ is a random congestion factor. Find the fastest route from the bottom-left node to the top-right node. Compare the basic Ant System, where every ant deposits pheromone, with the MAX–MIN Ant System (MMAS), and study the heuristic weight β.

Algorithm

1. Initialise pheromone $\tau_{ij} = 1$ on every edge.
2. Each of $m = 20$ ants walks from the source, moving from node i to an unvisited neighbour j with probability $p_{ij} = \tau_{ij}^{\alpha}\eta_{ij}^{\beta} / \sum_{l \in N_i} \tau_{il}^{\alpha}\eta_{il}^{\beta}$, where $\eta_{ij} = 1/w_{ij}$. An ant that reaches a dead end is discarded.
3. Evaporate: $\tau \leftarrow (1 - \rho)\,\tau$ with $\rho = 0.1$.
4. Deposit Q/L on the edges of a path of length L. *Ant System:* every successful ant deposits. *MMAS:* only the iteration-best ant (even iterations) or the global-best ant (odd iterations) deposits.
5. MMAS only: clip τ to $[\tau_{max}/2n,\ \tau_{max}]$ with $\tau_{max} = Q/(\rho L_{best})$.
6. Run 60 iterations; compare the best path with Dijkstra's exact shortest path.

Program (important sections)

```
def ant_walk(tau, alpha, beta):
    path, visited, node = [source], {source}, source
    while node != target:
        cand = np.array([j for j in neighbours[node] if j not in visited])
        if len(cand) == 0:
            return None, np.inf # dead end: ant is lost
        weights = tau[node, cand]**alpha * (1 / W[node, cand])**beta
        node = int(rng.choice(cand, p=weights / weights.sum()))
        path.append(node); visited.add(node)
    return path, sum(W[a, b] for a, b in zip(path, path[1:]))

# one iteration (MMAS)
tau *= (1 - rho) # evaporation
p, L = (it_path, it_len) if it % 2 == 0 else (best_path, best_len)
deposit(tau, p, Q / L) # best ant only
tau_max = Q / (rho * best_len)
tau[edges] = np.clip(tau[edges], tau_max / (2 * n_nodes), tau_max)
```

Key parameter settings: 20 ants, 60 iterations, $\alpha = 1$, $\beta = 1$, $\rho = 0.1$, $Q = 1$.

Output and Analysis

Program execution:

```
nodes = 40, edges = 99, source = 38, target = 7
Dijkstra shortest path: [38, 0, 2, 17, 8, 22, 13, 35, 7] length = 2.0681
ACO best path : [38, 0, 2, 17, 8, 22, 13, 35, 7] length = 2.0681
```

```
ACO found the optimum: True
Optimum first found at iteration 15
Ants on the best path in the final iteration: 78%
Execution Status: Successfully Executed
```

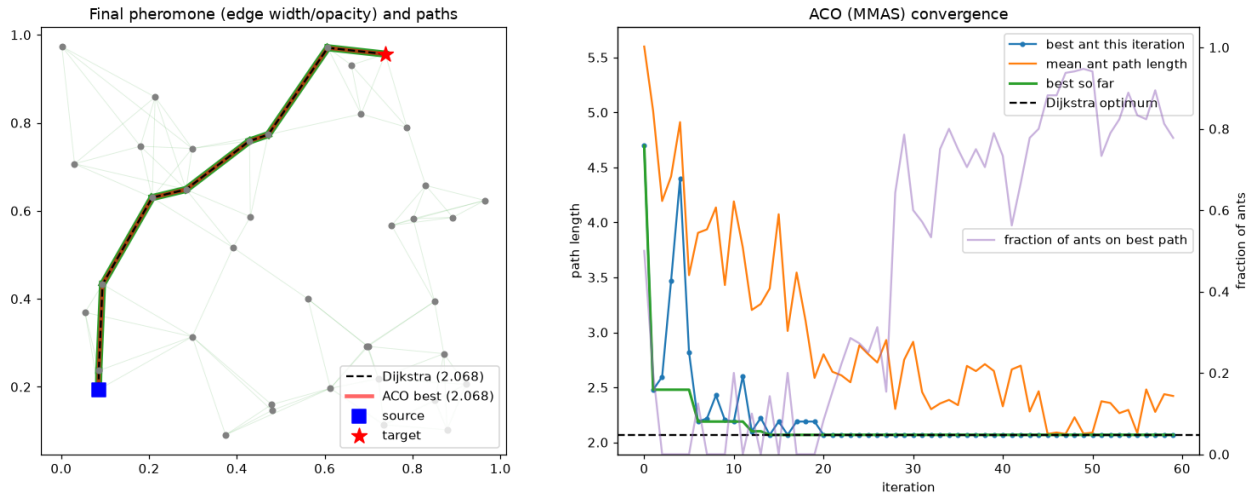


Figure 9: Left: final pheromone (edge width and opacity), the ACO path (red) and Dijkstra's path (dashed). Right: path lengths per iteration and the fraction of ants on the best path.

Parameter modification (runs reaching the Dijkstra optimum, out of 10 seeds):

S.No.	Parameter	Original	Modified	Original result	Modified result	Observation
1	Pheromone update	MMAS	Ant System	9/10 optimal	2/10 optimal	When every ant deposits, popular mediocre routes crowd out the best one.
2	Heuristic weight β	1	0	9/10 optimal	3/10 optimal	Ants that ignore edge costs wander; 35% get lost at dead ends.
3	Heuristic weight β	1	3	9/10 optimal	1/10 optimal	Greedy ants take the cheapest next edge, even into a detour.

Conclusion. MMAS finds the exact Dijkstra optimum (length 2.068, 8 edges) by iteration 15, and by the end 78% of the ants follow it along one strong pheromone trail. The basic Ant System struggles because its reward depends on how *many* ants walk an edge rather than how short their paths are: a 10% longer path earns only 10% less pheromone per ant. MMAS rewards quality directly, and its lower bound τ_{min} keeps alternative edges alive so the colony does not lock in early. $\beta \approx 1$ balances the colony's learned memory against local greed. On a static graph Dijkstra is exact and faster; ACO's strength is that it needs no global view and adapts when edge costs change (e.g. network routing).

Cross-Experiment Summary

Exp.	Method	Key takeaway
8	Simulated Annealing	Escapes the local minimum that traps hill climbing; cooling rate $0.80 \to 0.95$ raises success $80\% \to 100\%$.
9	Genetic Algorithm (TSP)	OX crossover + inversion mutation + elitism give a tour 9% shorter than nearest-neighbour.
10	PSO (Rosenbrock)	Solves 2-D to $f \approx 10^{-11}$; gbest PSO stalls in 10-D ($f \approx 3$), the known valley weakness.
11	L1 / L2 / Elastic Net	All penalties beat OLS for $n = 60$, $p = 40$; only L1 sets coefficients exactly to zero.
12	Kernel PCA	RBF Kernel PCA makes circles and moons linearly separable (100% / 99.7%); γ is critical.
13	ACO (shortest path)	MMAS matches Dijkstra in 9/10 runs vs 2/10 for the basic Ant System.